Devless

AI-Powered Code Generation from GitHub Issues

An army of AI workers at your service

The Business Problem

What I wanted

- **Technical PO capability** — create issues, get working code
- **Scalable workforce** — handle multiple issues in parallel
- **Review-ready PRs** — not just code dumps, but mergeable changes
- **Feedback loop** — respond to PR comments, iterate

The dream

> Open an issue → AI implements it → PR opens → Review → Merge

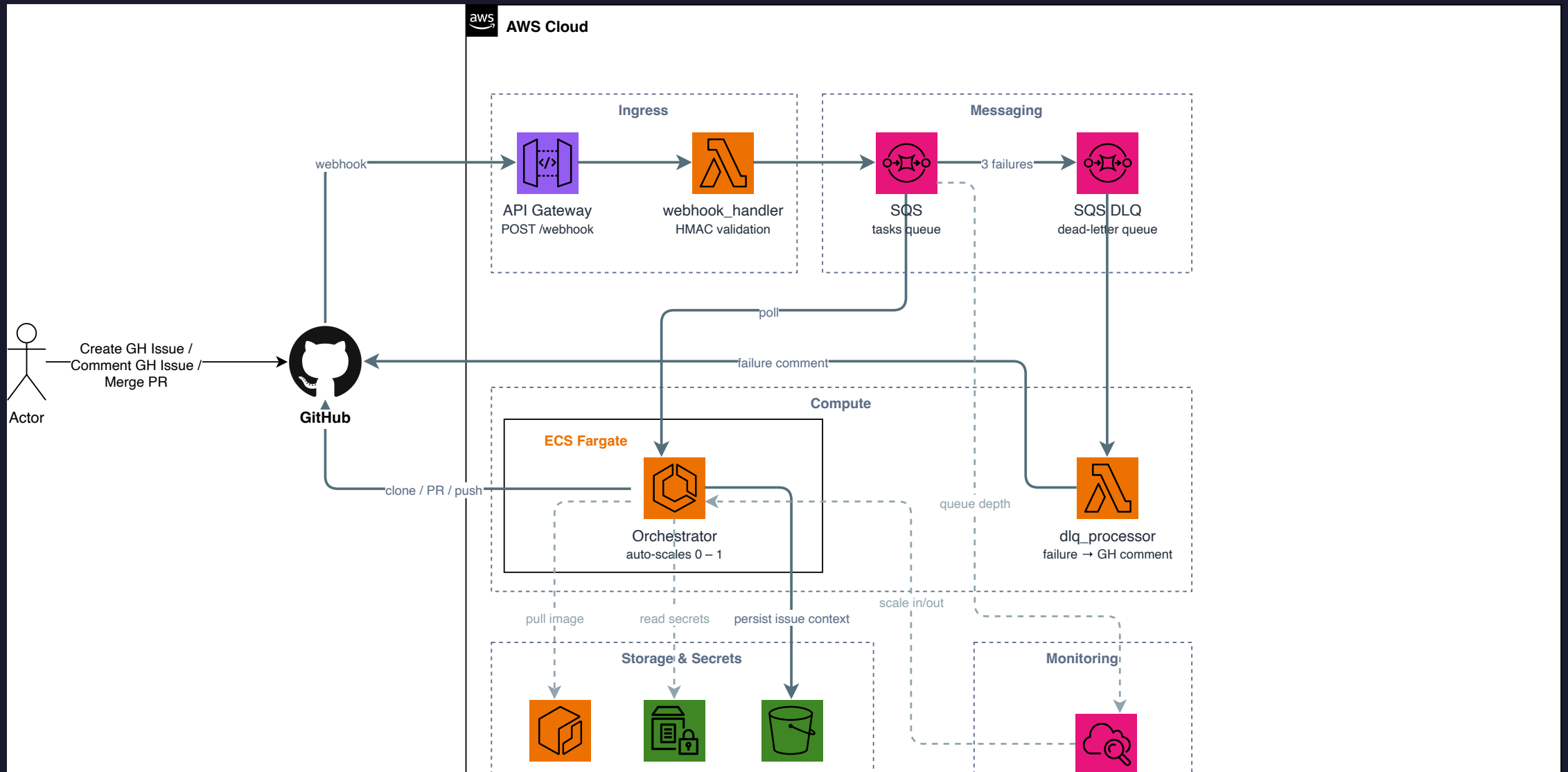
No context switching. No waiting. Just describe what you need.

Available Tools (3rd Party)

Tool	Approach	Pros	Cons
GitHub Copilot Workspace	Issue → PR	Native GitHub integration	Limited availability, closed
Devin	Autonomous agent	Full autonomy	$500/mo, black box
Sweep AI	Issue → PR	Simple setup	Limited customization
CodeRabbit	PR review only	Good reviews	Doesn't generate code
Aider	CLI coding assistant	Open source, flexible	No GitHub integration

Gap: Nothing that's open, customizable, AND integrates with our AWS sandbox.

The Solution: Devless



Key Components

Webhook Handler (Lambda)

- Validates GitHub HMAC signatures
- Filters bot events (prevents loops!)
- Routes: issues, PR comments, PR reviews

Orchestrator (ECS Fargate)

- State machine: `pending → cloned → agent_loop → pr_created → complete`
- Checkpoints to S3 (resume on failure)
- Runs Aider with Bedrock backend

Aider Integration

State Machine

(See sequence diagram in README.md)

States: `pending → cloned → agent_loop → pr_created → complete`

- **pending** — waiting to start
- **cloned** — repo cloned, branch created
- **agent_loop** — Aider generating code, running tests
- **pr_created** — branch pushed, PR opened
- **complete** — success comment posted

Demo Time

Live Demo

Repository: github.com/typekpb/devless-demo

Deployed App: typekpb.github.io/devless-demo

What we'll see:

1. Open an issue describing a feature
2. Devless picks it up, posts status
3. PR opens with implementation
4. Review, comment, iterate
5. Merge

Target Repo Setup

Minimal config in `.devless/config.yml`:

```
model: bedrock/eu.anthropic.claude-sonnet-4-20250514-v1:0
test_command: npm test
max_iterations: 10
```

Webhook events needed:

- `Issues`, `Issue comments`
- `Pull request review comments`, `Pull request reviews`

That's it. No agents to install, no tokens in the repo.

Development Journey

Tools Used

Tool	Purpose	Cost
Kiro	Specs + vibe coding	$20/mo subscription
Bedrock	LLM backend (Claude)	AWS sandbox (free for us)
Aider	Code generation engine	Open source
Terraform	Infrastructure	Open source

Why Kiro?

- Spec-driven development worked well for architecture
- "Vibe coding" for rapid iteration
- Free tier credits ran out fast → subscription needed

Why Bedrock?

Advantages

- **AWS sandbox available** — no credit card, no approvals
- **IAM authentication** — no API keys to manage
- **Claude Sonnet 4** — excellent for code generation
- **Regional endpoints** — EU data residency

Aider Integration

```
aider --model bedrock/eu.anthropic.claude-sonnet-4-20250514-v1:0
```

Just works. No OpenAI key needed.

Running Locally

Test Aider directly

```
cd /tmp && git clone https://github.com/typekpb/devless-demo.git
cd devless-demo

export AWS_REGION=eu-central-1
aider --model bedrock/eu.anthropic.claude-sonnet-4-20250514-v1:0 \
  --message "Add a dark mode toggle"
```

Run the orchestrator

```
cd orchestrator
pip install -e .
export SQS_QUEUE_URL=... S3_BUCKET=... GITHUB_APP_ID=...
python -m main
```

Lessons Learned

What Worked Well ✅

- **Aider** — battle-tested, handles edge cases
- **State machine** — clean separation, easy to debug
- **Checkpoints** — resume after failures, no lost work
- **GitHub App auth** — short-lived tokens, no PATs
- **Bedrock** — zero config with IAM roles

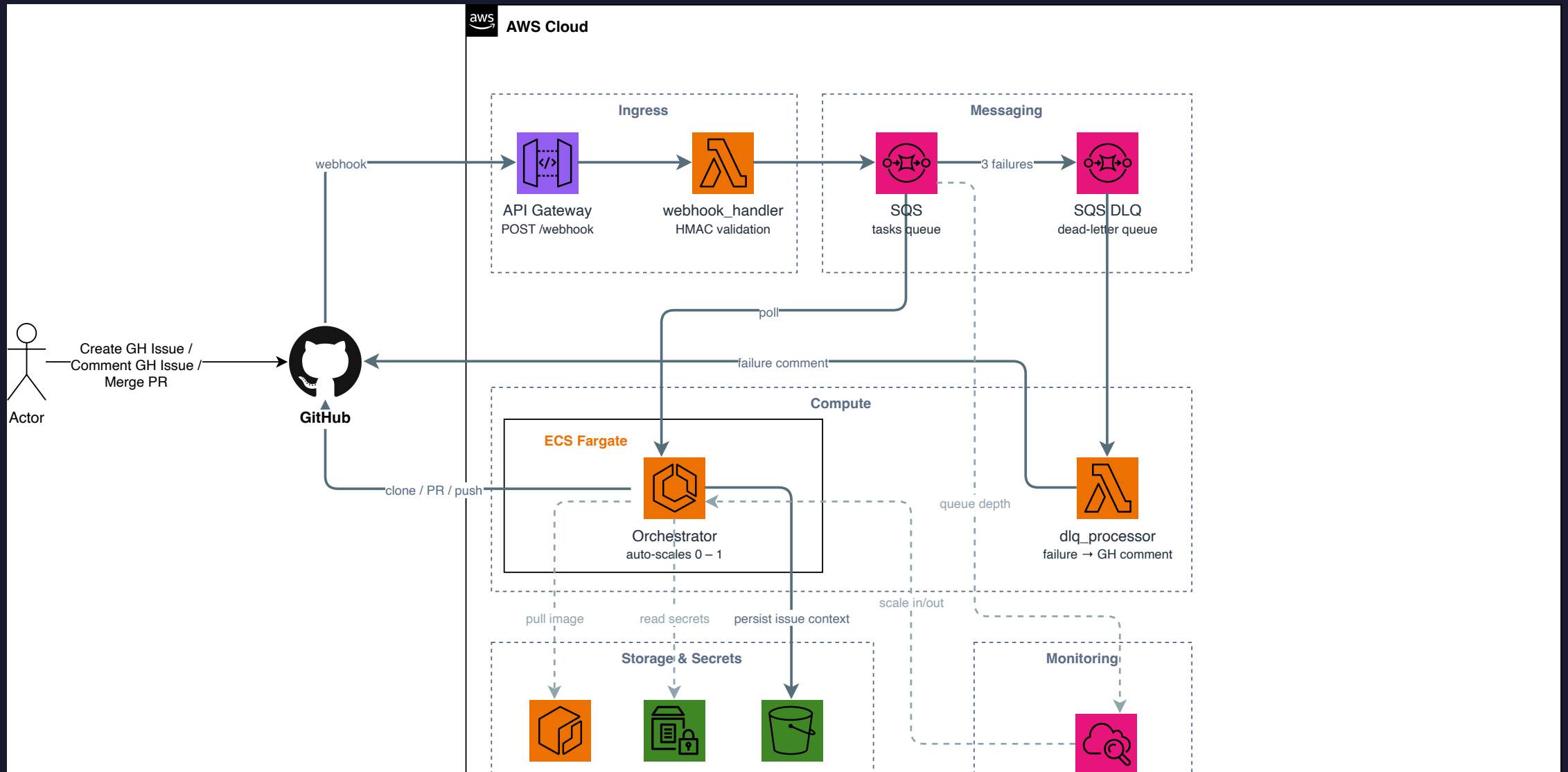
What I'd Do Differently 🔄

- Start with simpler event handling (disabled CI auto-fix)
- More upfront time on prompt engineering
- Consider LangGraph for complex workflows (future)

What I Didn't Have Time For

- **OpenAI / Azure backends** — code is there, not tested
- **Codex integration** — would be interesting to compare
- **Multi-repo orchestration** — one repo at a time for now
- **Cost optimization** — Bedrock pricing is fine, but could cache
- **Better error messages** — Aider output can be cryptic

Architecture Diagram



Infrastructure (Terraform)

```
terraform/
├── main.tf                 # Root module
├── modules/
│   ├── api_gateway/        # Webhook endpoint
│   ├── lambda/             # Webhook handler
│   ├── sqs/                # Task queue + DLQ
│   ├── ecs/                # Orchestrator service
│   ├── ecr/                # Container registry
│   ├── iam/                # Roles & policies
│   └── s3/                 # Checkpoints
└── terraform.tfvars        # Your config
```

One `terraform apply` deploys everything.

Cost Breakdown (Estimate)

Service	Usage	Cost/month
Lambda	~1000 invocations	~$0
SQS	~1000 messages	~$0
ECS Fargate	0.25 vCPU, 0.5GB	~$10
Bedrock (Claude)	~100 issues	~$5-20
S3	Checkpoints	~$0

Total: ~$15-30/month for moderate usage

Sandbox account = $0 for us

Future Ideas

- **LangGraph / Instructor** — smarter routing based on issue content
- **Multi-agent** — planner + coder + reviewer
- **PR auto-merge** — after approval + CI pass
- **Slack integration** — notifications, approvals
- **Cost tracking** — per-issue token usage
- **Fine-tuning** — on our codebase patterns

Q&A

Links

- **Devless repo:** github.com/typekpb/devless
- **Demo repo:** github.com/typekpb/devless-demo
- **Live demo:** typekpb.github.io/devless-demo
- **Aider:** aider.chat
- **Kiro:** kiro.dev

Backup: Event Flow Detail

(Refer to sequence diagram in README.md for detailed flow)

Summary:

1. GitHub Issue → API Gateway → Lambda (validate HMAC)
2. Lambda → SQS (buffer, retry)
3. ECS polls SQS → runs Aider with Bedrock
4. Aider generates code → ECS pushes branch → opens PR